This chapter is about sharing data safely between threads in C++, avoiding the potential problems that can arise, and maximizing the benefits.

3.1. Problems with sharing data between threads

When it comes down to it, the problems with sharing data between threads are all due to the consequences of modifying data. *If all shared data is read-only, there's no problem, because the data read by one thread is unaffected by whether or not another thread is reading the same data*. However, if data is shared between threads, and one or more threads start modifying the data, there's a lot of potential for trouble. In this case, you must take care to ensure that everything works out OK.

One concept that's widely used to help programmers reason about their code is that of *invariants*—statements that are always true about a particular data structure, such as "this variable contains the number of items in the list." These invariants are often broken during an update, especially if the data structure is of any complexity or the update requires modification of more than one value.

Consider a doubly linked list, where each node holds a pointer to both the next node in the list and the previous one. One of the invariants is that if you follow a "next" pointer from one node (A) to another (B), the "previous" pointer from that node (B) points back to the first node (A). In order to remove a node from the list, the nodes on either side have to be updated to point to each other. Once one has been updated, the invariant is broken until the node on the other side has been updated too; after the update has completed, the invariant holds again.

The steps in deleting an entry from such a list are shown in figure 3.1:

1. Identify the node to delete (N).

2. Update the link from the node prior to N to point to the node after N.